Translated from Chinese (Simplified) to English - www.onlinedoctranslator.com

Xstd

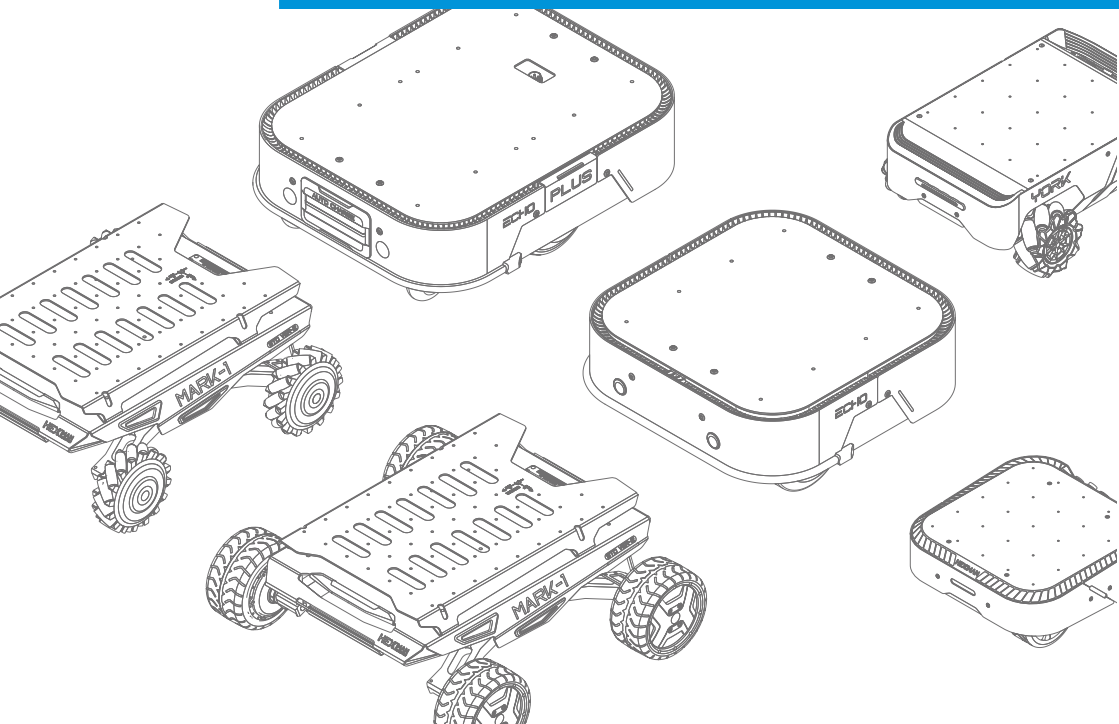
vehicle part

Standardized communication protocol

Chassis

User Manual

v1.1.1



Important security information

- Please confirm the operating voltage, power, installation and other parameters of the equipment before use. Problems caused by exceeding the parameters are not covered by the warranty.

- Please assess the IP rating, temperature, and equipment suitability requirements of the environment in which you will be using the equipment. Problems caused by harsh environments are not covered under warranty.

This device does not provide stable safety features such as collision avoidance, drop protection, or bio-proximity warning. Please conduct a safety assessment of the integrated product.

The assessment must be conducted in accordance with relevant regulations and certifications to ensure that the developed products do not pose any major safety hazards.

Please read the relevant maintenance requirements to prevent irreversible damage caused by incorrect operation, such as over-discharging the battery or low tire pressure.

Low pressure can cause damage to the wheel hub, and lack of lubrication can lead to axle wear.

- For first-time use, please place the device in a safe, open environment without any loads, start it according to the operating instructions, and test all phases.

To disable this function, please contact the service personnel of the purchasing channel if you have any questions.

- In case of equipment malfunction or accident, please immediately use the emergency stop or turn off the power to avoid secondary damage, and contact relevant technical support.

Personnel should not disassemble it themselves.

Note: Damage caused by user-initiated disassembly, modification, abnormal use, or force majeure events such as natural disasters will not be covered under warranty.

This product assumes no responsibility for any safety incidents caused by user-integrated devices. Users are advised to conduct their own risk assessments and reliability tests.

Version update information

V1.0.0: Starting version

V1.0.1: Updated chassis status settings and added braking function. V1.0.2:

Added zero-point calibration command.

V1.1.0: Added setting numbering protocol, modified error codes, and added special function control. V1.1.1: Added chassis error

information protocol, added emergency stop switch alarm, and modified chassis status information feedback.

Table of contents

1. Agreement Introduction.....5
1.1 Overview of Architecture.....5
1.2 Equipment Status Definition.....5
1.3 Equipment Classification and Identification.....5
1.4 Instruction Types.....5
2. CAN Communication Protocol.....6
2.1 CAN Data Structure.....6
2.2 Instruction Set Lookup Description.....7
2.3 CAN Command Directory.....8
2.4 General Equipment Instructions.....9
2.4.1 Device Reboot/Preparing to Reboot.....9
2.4.2 Software Version Inquiry.....10
2.4.3 General Settings / General Settings Successful.....11
2.4.4 Special State Reset / Special Reset Successful.....12
2.4.5 Motion State Reset / Motion Reset Successful.....13
2.4.6 Numbering Setup / Numbering Setup Successful.....14
2.4.7 Equipment Heartbeat Packet.....15
2.5 Equipment-Specific Instructions.....16
2.5.1.1 Chassis Status Settings / Chassis Status Information.....16
2.5.2 Chassis Motion Control / Chassis Motion Information.....18
2.5.3 Odometer Feedback.....19
2.5.4.1 Remote Control Information Transmission Enable / Remote Control Information.....20
2.5.4.2 Toggle Switch Position Table.....21
2.5.5.1 Safety Sensor Information Transmission Enable / Safety Sensor Information.....22
2.5.5.2 Touch Switch Table.....22
2.5.6 Driver Motion Information Enable / Driver Motion Information.....23
2.5.7.1 Driver Status Information Enable / Driver Status Information.....24
2.5.7.2 Driver Special Status Table.....24
2.5.8 Chassis Zero Point Calibration.....25
2.5.9 Chassis Error Messages.....26
3. Serial Communication Protocol.....28
3.1.1 Basic Serial Port Parameters.....28
3.1.2 Serial Port Frame Structure.....28
3.2 Serial Communication Case Study.....29
3.2.1 Case Chassis ID.....29
3.2.2 Serial Port Sending to Device Example Table.....29
3.2.3 Serial Port Slave Device Acceptance Case Table.....29
4. Software Support.....30
4.1 XVIEW Test System.....30
4.1 XROS Support.....31

1.1 Framework Introduction

XSTD is a communication protocol specification based on CAN networks and scalable to serial ports. This specification is designed to support multi-device and module designs. The design, while maintaining simplicity and ease of understanding, provides ample room for expansion for each module and supports multiple identical devices working online simultaneously. protocol Using a unified standard for similar devices sacrifices a small amount of efficiency for great convenience, which is very helpful for early feature development and learning.

1.2 Equipment Status Definition

Enable (default state of communication devices)

- All functions of the equipment are operating normally.
- Provide feedback on CAN and COM commands.

Secondary feedback is disabled by default, but can be enabled via command.

Disabled (default state for normal devices)

- The device enters a communication silence state, but its internal functions continue to operate.

Only general commands are accepted; special commands are not accepted.

- Only send heartbeat feedback and general command feedback, not special command feedback.
- Control devices installed on the device itself, such as radio frequency remote controllers, remain effective and have higher priority.

1.3 Classification and Identification of Equipment

Each device has three main identifying attributes: category, model, and serial number.

- The main categories are divided into 30 categories based on device type, with codes ranging from 0x01 to 0x1E, and 0x1F used as the broadcast code.
- The model number is a serial number from 0x01 to 0xFE. Within the same category, the model number is specific and unique, and is bound to the device name. 0xFF As a broadcast code.
- The serial number is from 0x01 to 0xFE, which can be set in XVIEW to distinguish devices of the same category and model. 0xFF is used for... For broadcast encoding.

When using the instructions, you need the corresponding category, model, and number. Please refer to the user manual of the corresponding device for the corresponding category and model code.

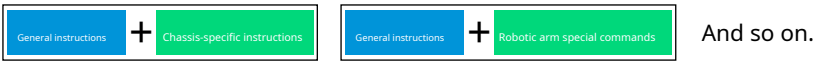
1.4 Instruction Types

Instructions are divided into general instructions and special instructions.

- General commands distinguish receiving devices only by CAN ID; the data content is the same for all major categories and is mainly used for basic functions. Settings and Feedback

Specialized instructions are categorized into major categories, each with different functions. Within the same major category, functions are selected as needed.

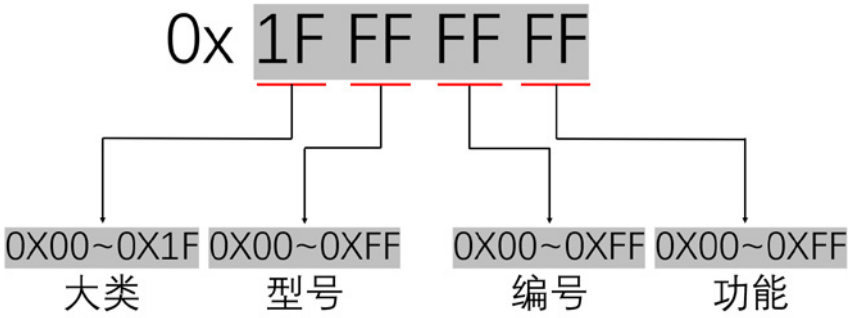
- The function code for general instructions starts from 0x01, and the corresponding feedback starts from 0xA1 (heartbeat feedback is 0xB0).
- The function codes for special instructions start from 0x11, and the corresponding feedback starts from 0xB1.



2.1 CAN Data Structure

The product uses CAN2.0B (extended frame) for CAN communication. The communication baud rate is 500K.

CAN ID encoding standard:



CAN data bits: Byte[0]~Byte[7]:

Byte[0]	Byte[1]	Byte[2]	Byte[3]	Byte[4]	Byte[5]	Byte[6]	Byte[7]

The CAN information used by CAN is the CANID and CAN_DATA mentioned above, totaling 12 bytes of data. When using, the needle Perform corresponding CAN_DATA data transmission for different CANIDs.

In CANID, the category and model are fixed parameters of the device and their values will not change. The number defaults to 1. When multiple devices of the same type are used at the same time, they can be modified through general commands.

CAN data frames					
Data bits	ID[3]	ID[2]	ID[1]	ID[0]	DATA[0~7]
Data types	unsigned int8	unsigned int8	unsigned int8	unsigned int8	8 x unsigned int8
Data Description	Major Categories	model	serial number	Function	8-byte CAN data

Chassis series fixed category 01

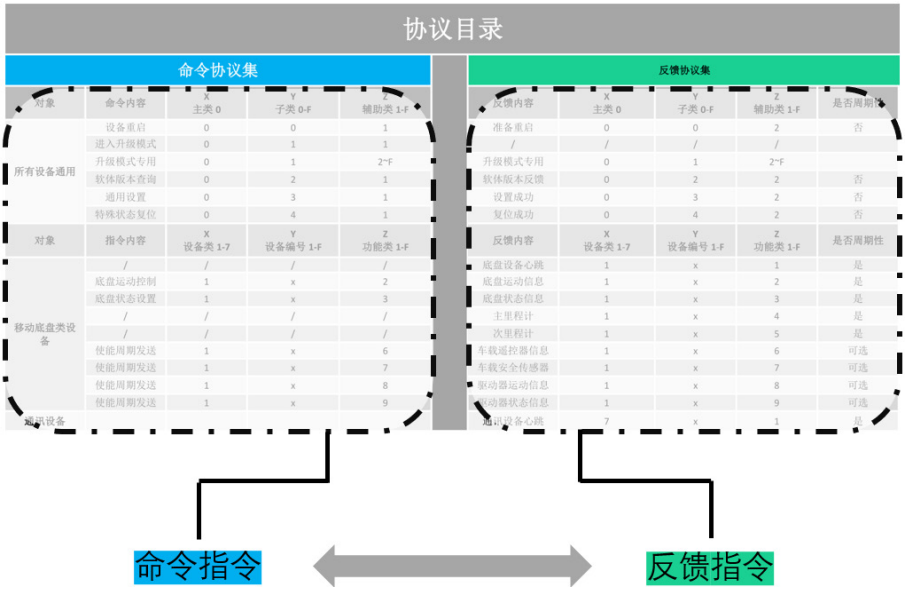
The default number is 01

The basic CANID format for chassis series is 01 XX 01 XX

2.2 Instruction Set Lookup Instructions

The CAN protocol is divided into general device commands and device-specific commands. General commands are applicable to all modules and components of the system and are commonly used for device startup, system upgrades, version checks, system resets, etc. Device-specific commands are specific to different device modules, and different modules can select and use certain dedicated commands as needed. Customized commands can be made for special devices.

The CAN closed-loop control system has a response mechanism for the vast majority of commands. That is, there is a one-to-one correspondence between command frames and feedback frames. The accuracy of the data stream is monitored in real time based on the feedback frames.



2.3 CAN Command Directory

Command Protocol Set						Feedback Protocol Set					
object	Command content	Major Categories 00~1F	model 00~FF	serial number 00~FF	Function 00~FF	Command content	Major Categories 00~1F	model 00~FF	serial number 00~FF	Function 00~FF	Periodic
All settings General purpose	Device restart	01	XX	XX (default 01)	01	Preparing to restart	01	XX	XX (default 01)	A1	no
	Software version query	01	XX	XX (default 01)	02	Software version feedback	01	XX	XX (default 01)	A2	no
	General Settings	01	XX	XX (default 01)	03	General settings successful	01	XX	XX (default 01)	A3	no
	Special state reset	01	XX	XX (default 01)	04	Special reset successful	01	XX	XX (default 01)	A4	no
	Motion state reset	01	XX	XX (default 01)	05	successful motion reset	01	XX	XX (default 01)	A5	no
	Numbering settings	01	XX	XX (default 01)	06	Numbering successfully set	01	XX	XX (default 01)	A6	no
	/	/	/	/	/	Heartbeat Pack	01	XX	XX (default 01)	B0	yes
object	Command content	Major Categories 00~1F	model 00~FF	serial number 00~FF	Function 00~FF	Command content	Major Categories 00~1F	model 00~FF	serial number 00~FF	Function 00~FF	Periodic
move chassis	Chassis status settings	01	XX	XX (default 01)	11	Chassis status information	01	XX	XX (default 01)	B1	yes
	Chassis motion control	01	XX	XX (default 01)	12	Chassis motion information	01	XX	XX (default 01)	B2	yes
	/	/	/	/	/	Main odometer	01	XX	XX (default 01)	B3	yes
	/	/	/	/	/	Secondary odometer	01	XX	XX (default 01)	B4	yes
	Remote control information transmission enable	01	XX	XX (default 01)	15	Car remote control information	01	XX	XX (default 01)	B5	After enabling, it is
	Security sensor sends enable	01	XX	XX (default 01)	16	Vehicle safety sensor information	01	XX	XX (default 01)	B6	After enabling, it is
	Driver motion information transmission enable	01	XX	XX (default 01)	17	Driver motion information	01	XX	XX (default 01)	B7	After enabling, it is
	Driver status information transmission enable	01	XX	XX (default 01)	18	Driver status information	01	XX	XX (default 01)	B8	After enabling, it is
	Chassis zero-point calibration	01	XX	XX (default 01)	19	Zero point calibration information	01	XX	XX (default 01)	B9	no

2.4 General Equipment Instructions

2.4.1 Device restart/Preparing to restart

Device restart				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 01	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Major Categories	unsigned int8	Constantly 0x01	
byte[1]	model	unsigned int8	Consult the device user manual for details.	
byte[2]	serial number	unsigned int8	The default value is 0x01	
byte[3~7]	/	/	/	

Note: When multiple machines are used simultaneously in the system, the device number can be changed. The default device number is 0x01. The length of the sent command data is 8 bits, with invalid bits padded with 0 (the same applies below).

Preparing to restart				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX A1	0	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0~7]	/	/	/	

Note: The device restart command restarts the system. After receiving the command, the device prepares to restart and responds. The system restarts, and the buzzer sounds three times after two seconds, indicating that the restart is complete.

Example: Send CANID: 0x01 02 03 01 and data: 01 02 03 00 00 00 00 to chassis with model number 0x02 and part number 0x03, and the system will restart.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 01	Data Frame	Extended Frame	8	01 02 03 00 00 00 00 00
host computer	0x01 02 03 A1	Data Frame	Extended Frame	0	/

2.4.2 Software Version Inquiry

Software version query				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 02	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0~7]	/	/	/	

Software version feedback				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX A2	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Hardware version character [0]	unsigned int8 ASCII	/	
byte[1]	Hardware version character [1]	unsigned int8 ASCII	/	
byte[2]	Hardware version character [2]	unsigned int8 ASCII	/	
byte[3]	Hardware version character[3]	unsigned int8 ASCII	/	
byte[4]	Hardware version character[4]	unsigned int8 ASCII	/	
byte[5]	Hardware version character[5]	unsigned int8 ASCII	/	
byte[6]	Hardware version character[6]	unsigned int8 ASCII	/	
byte[7]	Hardware version character[7]	unsigned int8 ASCII	/	

Note: Read the system version number.

Example: Send CANID: 0x01 02 03 02 and data: 00 00 00 00 00 00 00 00 to chassis with model number 0x02 and part number 0x03.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 02	Data Frame	Extended Frame	8	00 00 00 00 00 00 00 00
host computer	0x01 02 03 A2	Data Frame	Extended Frame	8	XX XX XX XX XX XX XX XX

2.4.3 General Settings / General Settings Successful

General Settings				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x01 XX XX 03	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Major Categories	unsigned int8	Constantly 0x01	
byte[1]	model	unsigned int8	Consult the device user manual for details.	
byte[2]	serial number	unsigned int8	The default value is 0x01	
byte[3]	Enable	unsigned int8	0x00: Power off device / 0x01: Power on device	
byte[4~7]	/	/	/	

General settings successful				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX A3	0	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0~7]	/	/	/	

Note: General settings commands are used to enable the device.

The chassis is disabled by default when powered on.

Example: To enable the chassis with device model 0x02 and serial number 0x03, send the command 01 02 03 01 00 00 00 00 to system ID: 0x01 02 03 03; the device is enabled.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 03	Data Frame	Extended Frame	8	01 02 03 01 00 00 00 00
host computer	0x01 02 03 A3	Data Frame	Extended Frame	0	/

2.4.4 Special State Reset / Special Reset Successful

Special state reset				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 04	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	State clearing	unsigned int8	0xCC: Clear / No action taken for other values	
byte[1~7]	/	/	/	

Special reset successful				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX A4	0	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0~7]	/	/	/	

Note: Special state reset is used to clear and reset the system when an error occurs.

Example: Send CANID: 0x01 02 03 04 and data: CC 00 00 00 00 00 00 00 to chassis with model number 0x02 and part number 0x03.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 04	Data Frame	Extended Frame	8	CC 00 00 00 00 00 00 00
host computer	0x01 02 03 A4	Data Frame	Extended Frame	0	/

2.4.5 Motion State Reset / Motion Reset Successful

Special state reset				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 05	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Clear motion status	unsigned int8	0xCC: Clear / No action taken for other values	
byte[1~7]	/	/	/	

Special reset successful				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX A5	0	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0~7]	/	/	/	

Note: Motion status reset is used to reset the system's motion data (such as odometer data).

Example: Send CANID: 0x01 02 03 05 and data: CC 00 00 00 00 00 00 00 to chassis with model number 0x02 and part number 0x03.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 05	Data Frame	Extended Frame	8	CC 00 00 00 00 00 00 00
host computer	0x01 02 03 A5	Data Frame	Extended Frame	0	/

2.4.6 Numbering Settings

General Settings				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x01 XX XX 06	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Major Categories	unsigned int8	Constantly 0x01	
byte[1]	model	unsigned int8	Consult the device user manual for details.	
byte[2]	serial number	unsigned int8	The default value is 0x01	
byte[3]	New number	unsigned int8	If the number is not changed, it will be the same as the original number.	
byte[4~7]	/	/		
			/	

General settings successful				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x01 XX XX A6	0	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0~7]	/	/	/	

Note: The number setting command is used to assign a new number to a device. When the system has only a single device, the number always defaults to 0x01.

When there are multiple devices in the system, the device number can be customized.

For example: If the chassis with device model 0x02 and serial number 0x03 changes its device number to 0x02, the system ID 0x01 02 03 06 sends the instruction 01 02 03 02 00 00 00 00; the device number changes from 0x03 to 0x02.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 06	Data Frame	Extended Frame	8	01 02 03 02 00 00 00 00
host computer	0x01 02 03 A6	Data Frame	Extended Frame	0	/

2.4.7 Equipment Heartbeat Packet

Heartbeat Pack				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX B0	1	500
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Enable state	unsigned int8	0x00: Disabled / 0x01: Enabled	
byte[1~7]	/	/	/	

Note: Heartbeat packets are mainly used to confirm the online status of the device. They are sent every 500ms regardless of the status and are also the key frame for confirming whether the device is enabled.

Example: Model 0x02, Serial Number 0x03, Enabled chassis feedback heartbeat packet.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	/	/	/	/	/
host computer	0x01 02 03 B0	Data Frame	Extended Frame	1	01

2.5 Equipment-Specific Instructions

2.5.1 Chassis Status Settings / Chassis Status Information

Chassis status settings				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 11	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Set control mode	unsigned int8	0x00: Standby / 0x01: Remote Control / 0x02: CAN Control / 0x03: Free	
byte[1]	Buzzer Enable	unsigned int8	0: Off / 1: On (default)	
byte[2]	Enable braking	unsigned int8	0x00: Release (default if no brake) / 0x01: Lock (default if brake is present)	
byte[3]	Enable special functions	unsigned int8	0x00: Off (default) / 0x01: On	
byte[4~7]	/	/	/	

Chassis status information				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX B1	6	100
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Equipment status	unsigned int8	0x00: Normal / 0x01: Abnormal (Error alarm occurred)	
byte[1]	Control Mode	unsigned int8	0x00: Standby / 0x01: Remote Control / 0x02: CAN Control / 0x03: Free	
byte[2]	Battery voltage low 8 bits	unsigned int16	Unit: 0.1V	
byte[3]	Battery voltage high 8 bits			
byte[4]	Buzzer Enable State	unsigned int8	0: Off / 1: On (default)	
byte[5]	Remote control status	unsigned int8	0: Online / 1: Offline	
byte[6]	Braking status	unsigned int8	0x00: Release (default if no brake) / 0x01: Lock (default if brake is present)	
byte[7]	Special function status	unsigned int8	0x00: Off (default) / 0x01: On	

Note: When controlling the chassis via a wired connection, you must first switch the control mode to CAN mode. If using a remote control, please switch the SWA switch on the remote control to the bottom CAN mode. For remote control instructions, please refer to the device's user manual.

Brake enable only applies to chassis with braking function. If there is no braking function, this bit is kept at 0x00 by default. If there is braking function, the brake is enabled by default.

Special functions vary depending on the chassis. For example, the Ackermann chassis has a differential lock function, but not all chassis have special functions. Special functions can also be activated by remote control; please refer to the user manual of the corresponding device.

Example: Chassis model 0x02, part number 0x03 is changed to CAN mode control and the buzzer is enabled.

CANID: 0x 01 02 03 11 Data: 02 01 00 00 00 00 00 00.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 11	Data Frame	Extended Frame	8	02 01 00 00 00 00 00 00
host computer	0x01 02 03 B1	Data Frame	Extended Frame	7	00 02 XX XX 01 00 00

2.5.2 Chassis Motion Control / Chassis Motion Information

Chassis motion control				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 12	8	50 (Recommended value)
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	x-direction velocity (low 8 bits)	signed int16	Unit: mm/s	
byte[1]	x-direction velocity (high 8 bits)			
byte[2]	y-direction velocity (low 8 bits)※	signed int16	Unit: mm/s	
byte[3]	y-direction velocity (high 8 bits)※			
byte[4]	Angular velocity (low 8 bits)	signed int16	Unit: 0.001 rad/s	
byte[5]	Angular velocity (high 8 bits)			
byte[6~7]	/	/	/	

Chassis motion information				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX B2	6	20
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	x-direction velocity (low 8 bits)	signed int16	Unit: mm/s	
byte[1]	x-direction velocity (high 8 bits)			
byte[2]	y-direction velocity (low 8 bits)※	signed int16	Unit: mm/s	
byte[3]	y-direction velocity (high 8 bits)※			
byte[4]	Angular velocity (low 8 bits)	signed int16	Unit: 0.001 rad/s	
byte[5]	Angular velocity (high 8 bits)			
byte[6~7]	/	/	/	

Note 1. The Y-direction speed is only effective for vehicles with translation capabilities, such as those with Mecanum wheel systems. For other vehicles, the default value is 0.

2. Due to the inherent precision limitations of motor operation, the system feedback value may slightly deviate from the set value.

3. Data uses signed variables and conforms to the C language standard.

Example: Chassis with model number 0x02 and part number 0x03 moves forward at a speed of 0.5m/s while rotating at an angle of -0.1rad/s, sending CANID: 0x01 02 03 12 and data: F4 01 00 00 9C FF 00 00.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 12	Data Frame	Extended Frame	8	F4 01 00 00 9C FF 00 00
host computer	0x01 02 03 B2	Data Frame	Extended Frame	6	F4 01 00 00 9C FF

2.5.3 Odometer Feedback

Main odometer				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX B3	8	20
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Left wheel odometer [7:0]	signed int32	The average mileage is calculated for the left side in the four-wheel differential system. Unit: mm	
byte[1]	Left wheel odometer [15:8]			
byte[2]	Left wheel odometer [23:16]			
byte[3]	Left wheel odometer [31:24]			
byte[4]	Right wheel odometer [7:0]	signed int32	In a four-wheel differential system, the average mileage is on the right side. Unit: mm	
byte[5]	Right wheel odometer [15:8]			
byte[6]	Right wheel odometer [23:16]			
byte[7]	Right wheel odometer [31:24]			

Secondary odometer				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX B4	8	20
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Left rear wheel odometer [7:0]②	signed int32	Applicable to omnidirectional wheel and Mach wheel systems Unit: mm	
byte[1]	Left rear wheel odometer [15:8]②			
byte[2]	Left rear wheel odometer [23:16]②			
byte[3]	Left rear wheel odometer [31:24]②			
byte[4]	Right rear wheel odometer [7:0]②	signed int32	Applicable to omnidirectional wheel and Mach wheel systems Unit: mm	
byte[5]	Right rear wheel odometer [15:8]②			
byte[6]	Right rear wheel odometer [23:16]②			
byte[7]	Right rear wheel odometer [31:24]②			

Note: Two-wheel differential and four-wheel differential systems use only the master odometer; omnidirectional wheel and Mecanum wheel systems use two sets of odometers simultaneously.

Example: Chassis feedback odometer data for model 0x02, serial number 0x03.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	/	/	/	/	/
host computer	0x01 02 03 B3	Data Frame	Extended Frame	8	XX XX XX XX XX XX XX XX

2.5.4.1 Remote Control Information Transmission Enable/Remote Control Information

Remote control information transmission enable				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 15	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Enable cycle sending	unsigned int8	0: Off (default) / 1~20: 20ms / 21~255: Setting value	
byte[1~7]	/	/	/	

Remote control information				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX B5	7	20~255 (Off by default)
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	toggle switch	unsigned int8	According to the toggle switch position table	
byte[1]	Left joystick horizontal axis	signed int8	Range: -100~100	
byte[2]	Left joystick vertical axis	signed int8	Range: -100~100	
byte[3]	Right joystick horizontal axis	signed int8	Range: -100~100	
byte[4]	Right stick vertical axis	signed int8	Range: -100~100	
byte[5]	Left knob VRA	signed int8	Range: -100~100	
byte[6]	Right knob VRA	signed int8	Range: -100~100	
byte[7]	/	/	/	

Note: Feedback is only available for the official vehicle-mounted RF remote control. The vehicle-mounted RF remote control is an optional device; please contact customer service for details.

2.5.4.2 Toggle Switch Position Table

Toggle switch position gauge		
Bit	State value	Status Description
bit[0]	0: Failed / 1: Upgraded	SWA
bit[1]	2: Medium / 3: Low	
bit[2]	0: Failed / 1: Upgraded	SWB
bit[3]	2: Medium / 3: Low	
bit[4]	0: Failed / 1: Upgraded	SWC
bit[5]	2: Medium / 3: Low	
bit[6]	0: Failed / 1: Upgraded	SWD
bit[7]	2: Medium / 3: Low	

Example: Chassis with model 0x02 and serial number 0x03 is set to provide remote control feedback within 100ms. CANID: 0x01 02 03 15 Data: 64 00 00 00 00 00 00 00.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 15	Data Frame	Extended Frame	8	64 00 00 00 00 00 00 00
host computer	0x01 02 03 B5	Data Frame	Extended Frame	7	XX XX XX XX XX XX XX

2.5.5.1 Safety Sensor Information Transmission Enable/Safety Sensor Information

Security sensor sends enable				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 16	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Enable cycle sending	unsigned int8	0: Off (default) / 1~20: 20ms / 21~255: Setting value	
byte[1~7]	/	/	/	

Safety sensor information				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX B6	1	20~255 (Off by default)
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Contact switch	unsigned int8	Numbering by bit	
byte[1~7]	/	/	/	

Note: This data is only reported by devices equipped with anti-collision strips or touch sensors.

2.5.5.2 Touch Switch Meter

Touch switch meter		
Bit	State value	Status Description
bit[0]	0: Not triggered / 1: Triggered	Touch switch [0]
bit[1]	0: Not triggered / 1: Triggered	Touch switch [1]
bit[2]	0: Not triggered / 1: Triggered	Touch switch [2]
bit[3]	0: Not triggered / 1: Triggered	Touch switch [3]
bit[4]	0: Not triggered / 1: Triggered	Touch switch [4]
bit[5]	0: Not triggered / 1: Triggered	Touch switch [5]
bit[6]	0: Not triggered / 1: Triggered	Touch switch [6]
bit[7]	0: Not triggered / 1: Triggered	Touch switch [7]

Example: Chassis with model number 0x02 and serial number 0x03 is set to provide 100ms safety feedback, sending CANID: 0x01 02 03 16 and data: 64 00 00 00 00 00 00 00.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 16	Data Frame	Extended Frame	8	64 00 00 00 00 00 00 00
host computer	0x01 02 03 B6	Data Frame	Extended Frame	1	XX

2.5.6 Driver Motion Information Enable / Driver Motion Information

Driver motion information enable				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 17	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Enable cycle sending	unsigned int8	0: Off (default) / 1~20: 20ms / 21~255: Setting value	
byte[1~7]	/	/	/	

Driver motion information				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX B7	7	20~255 (Off by default)
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Driver Number	unsigned int8	0~7 (sent in turn)	
byte[1]	Low 8 bits of speed	signed int16	Unit: PRM	
byte[2]	High speed 8 bits			
byte[3]	Position value [7:0]	signed int32	Unit: number of pulses	
byte[4]	Position value [15:8]			
byte[5]	Position value [23:16]			
byte[6]	Position value [31:24]			
byte[7]	/	/	/	

Note: This data is a direct count from the driver at the lowest system level. Users can calculate the odometer reading and speed themselves based on the raw data.

Example: A chassis with model number 0x02 and part number 0x03 is set to provide 100ms driver motion feedback, sending CANID: 0x01 02 03 17 and data: 64 00 00 00 00 00 00 00.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 17	Data Frame	Extended Frame	8	64 00 00 00 00 00 00 00
host computer	0x01 02 03 B7	Data Frame	Extended Frame	7	XX XX XX XX XX XX XX

2.5.7.1 Driver Status Information Enable / Driver Status Information

Driver status information enable				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 18	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Enable cycle sending	unsigned int8	0: Off (default) / 1~20: 20ms / 21~255: Setting value	
byte[1~7]	/	/	/	

Driver status information				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX B8	8	20~255 (Off by default)
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Driver Number	unsigned int8	0~7 (sent in turn)	
byte[1]	Low 8 bits of voltage	unsigned int16	Unit: 0.1V	
byte[2]	High 8 bits of voltage			
byte[3]	Low 8 bits of current	unsigned int16	Unit: 0.1A	
byte[4]	High 8 bits of current			
byte[5]	Driver temperature	signed int8	Unit: 1°C	
byte[6]	Motor temperature	signed int8	Unit: 1°C	
byte[7]	Special states of the driver	unsigned int8	According to the driver special state table	

2.5.7.2 Driver Special Status Table

Touch switch meter		
Bit	State value	Status Description
bit[0]	0: Normal / 1: Too low	Drive low voltage alarm
bit[1]	0: Normal / 1: Overheating	Drive over-temperature alarm
bit[2]	0: Normal / 1: Overcurrent	Motor overcurrent alarm
bit[3]	0: Normal / 1: Overheating	Motor over-temperature alarm
bit[4~7]	/	/

Example: A chassis with model 0x02 and part number 0x03 is set to provide 100ms driver status feedback, sending CANID: 0x01 02 03 18 and data: 64 00 00 00 00 00 00 00.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 18	Data Frame	Extended Frame	8	64 00 00 00 00 00 00 00
host computer	0x01 02 03 B8	Data Frame	Extended Frame	8	XX XX XX XX XX XX XX XX

2.5.8 Chassis zero-point calibration

Chassis zero-point calibration				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
external	This equipment	0x 01 XX XX 19	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Zero offset 1 [7:0]	signed int32	Unit: mrad	
byte[1]	Zero offset 1 [15:8]			
byte[2]	Zero offset 1 [23:16]			
byte[3]	Zero offset 1 [31:24]			
byte[4]	Zero offset 2 [7:0]	signed int32	Unit: mrad	
byte[5]	Zero offset 2 [15:8]			
byte[6]	Zero offset 2 [23:16]			
byte[7]	Zero offset 2 [31:24]			

Zero point calibration information				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX B9	8	/
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Zero offset 1 [7:0]	signed int32	Unit: mrad	
byte[1]	Zero offset 1 [15:8]			
byte[2]	Zero offset 1 [23:16]			
byte[3]	Zero offset 1 [31:24]			
byte[4]	Zero offset 2 [7:0]	signed int32	Unit: mrad	
byte[5]	Zero offset 2 [15:8]			
byte[6]	Zero offset 2 [23:16]			
byte[7]	Zero offset 2 [31:24]			

Note: Use offset 1 for a single Ackerman; use 1 for forward steering and 2 for backward steering for both front and rear Ackerman steering wheels; use 1 for the left steering wheel and 2 for the right steering wheel for both front and rear Ackerman steering wheels.

Example: A single Ackerman chassis with model number 0x02 and serial number 0x03 is set to an offset of 1000mrad and sends CANID: 0x01 02 03 19 and data: E8 03 00 00 00 00 00 00.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	0x01 02 03 19	Data Frame	Extended Frame	8	E8 03 00 00 00 00 00 00
host computer	0x01 02 03 B9	Data Frame	Extended Frame	8	E8 03 00 00 00 00 00 00

2.5.9 Chassis Error Messages

Error status feedback				
sender	**Recipient**	**CAN ID**	**Data length**	**Period (ms)**
This equipment	external	0x 01 XX XX BA	4	500
Data bits	**Function**	**Data types**	**illustrate**	
byte[0]	Motor failure	unsigned int8	Check the motor fault table	
byte[1]	Drive failure	unsigned int8	Check the driver fault table	
byte[2]	Communication failure	unsigned int8	Check the communication fault table	
byte[3]	Other faults	unsigned int8	See other fault tables	
byte[4~7]	/	/	/	

Motor Fault Table		
Bit	State value	Status Description
bit[0]	0: Normal / 1: Faulty	Motor overcurrent alarm
bit[1]	0: Normal / 1: Faulty	Motor over-temperature alarm
bit[2]	0: Normal / 1: Faulty	Encoder failure
bit[3]	0: Normal / 1: Faulty	Hall sensor failure
bit[4~7]	/	/

Drive Fault Table		
Bit	State value	Status Description
bit[0]	0: Normal / 1: Faulty	Drive low voltage alarm
bit[1]	0: Normal / 1: Faulty	Drive over-temperature alarm
bit[2~7]	/	/

Communication Fault Table		
Bit	State value	Status Description
bit[0]	0: Online / 1: Offline	Driver No. 1 communication failure
bit[1]	0: Online / 1: Offline	Driver No. 2 communication failure
bit[2]	0: Online / 1: Offline	Communication failure of driver #3
bit[3]	0: Online / 1: Offline	Driver No. 4 communication failure
bit[4~7]	/	/

Other Fault Table		
Bit	State value	Status Description
bit[0]	0: Normal / 1: Warning	Low battery voltage warning (<15%)
bit[1]	0: Normal / 1: Faulty	Low battery voltage alarm (<5%)
bit[2]	0: Not triggered / 1: Triggered	Touch sensor trigger
bit[3]	0: Not triggered / 1: Triggered	Emergency stop switch
bit[4~7]	/	/

Example: The chassis feedback touch sensor with model number 0x02 and serial number 0x03 is triggered.

Send Instance					
Transmission target	CAN ID	Frame format	Frame type	Data length	Data bits (L->H)
equipment	/	/	/	/	/
host computer	0x01 02 03 BA	Data Frame	Extended Frame	4	00 00 00 04

3.1 Serial Communication Protocol

The serial communication protocol is a packaged encapsulation of the CAN protocol, adding a frame header, frame length, frame ID, and checksum. Instruction details Please refer to the section and use cases.CAN communication protocol section

3.1.1 Basic Serial Port Parameters

project	parameter
baud rate	460800
check	No verification
Data bit length	8
Stop bit	1

3.1.2 Serial Port Frame Structure

To simplify system design and improve system reusability, the serial port bytes 3-14 are kept consistent with the ten-bit data used in the CAN protocol. Start bit, frame length, and frame ID are set to default values.The checksum is the sum of the first 15 bytes, with the last byte being the result.

data.

Data Definition	HEAD	LEN	DATA[0-11]	FRAME_ID	SUM
Data types	Uint8	Uint8	12*Uint8	Uint8	Uint8
illustrate	0x55	0x10	12-bit CAN message data Serial port messages are for CAN Message encapsulation	Frame ID can be fixed or fixed. Accumulation, as a continuous number According to the sequence stamp	The accumulation of the first 15 items Sum and then take the least significant digit. Uint8 data

Serial port data bits	DATA[0]	DATA[1]	DATA[2]	DATA[3]	DATA[4~11]
CAN data bits	ID[3]	ID[2]	ID[1]	ID[0]	DATA[0~7]
Data types	Uint8	Uint8	Uint8	Uint8	8 x Uint8
Data Description	Major Categories	model	serial number	Function	8-byte CAN data

3.2 Serial Communication Case Study

The following uses the ECHO-PLUS chassis as an example to list commonly used chassis control commands.

3.2.1 Case Chassis ID

Chassis Fixing Category 01

Case Model 03

Case Number 05

Serial port demonstration case: CANID fixed format 01 03 05 XX

3.2.2 Serial Port Sending to Device Example Table

Start bit frame length		CANID[0~3]	CAN data [4~11]	Frame ID and Checksum		illustrate
55	10	01 03 05 01	01 03 05 00 00 00 00 00	01	79	Device restart
55	10	01 03 05 03	01 03 05 01 00 00 00 00	01	7C	Device enable
55	10	01 03 05 04	CC 00 00 00 00 00 00 00	01	3F	Reset special state
55	10	01 03 05 05	CC 00 00 00 00 00 00 00	01	40	Reset motion state
55	10	01 03 05 11	02 01 00 00 00 00 00 00	01	83	Set CAN mode
55	10	01 03 05 12	F4 01 00 00 00 00 00 00	01	76	Linear speed 0.5 m/s
55	10	01 03 05 12	00 00 00 00 DC 05 00 00	01	62	Rotational speed 1.5 rad/s
55	10	01 03 05 15	64 00 00 00 00 00 00 00	01	E8	Remote control feedback 100ms
55	10	01 03 05 17	32 00 00 00 00 00 00 00	01	B8	Driven motion feedback 50ms

3.2.3 Serial Port Slave Device Acceptance Case Table

Start bit frame length		CANID [0~3]	CAN data [4~11]	Frame ID and Checksum		illustrate
55	08	01 03 05 A1	/	01	08	Preparing to restart
55	09	01 03 05 B0	01	01	19	Heartbeat Pack
55	08	01 03 05 A4	/	01	0B	Special state reset successful
55	08	01 03 05 A5	/	01	0C	Motion state reset successful
55	0E	01 03 05 B1	00 02 09 01 01 00 00	01	2B	Chassis status information (24.5V)
55	0E	01 03 05 B2	F4 01 00 00 00 00	01	14. Chassis motion information (0.5m/s)	
55	10	01 03 05 B3	DC 05 00 00 D0 07 00 00	01	DA	Main odometer (1.5m on the left, 2m on the right)
55	0F	01 03 05 B5	03 00 00 00 00 00 00	01	26	Remote control information (under SWA)
55	0F	01 03 05 B7	02 64 00 33 44 55 00	01	57	Drive information for number 2 (100 RPM)

4.1 XVIEW Test System

This system is a human-computer interaction software designed for debugging and development. It can test the functions of various devices online and provide real-time feedback on device data. It can also be used for serial port and CAN bus monitoring and testing. This system requires the use of a CAN-COM HUB.

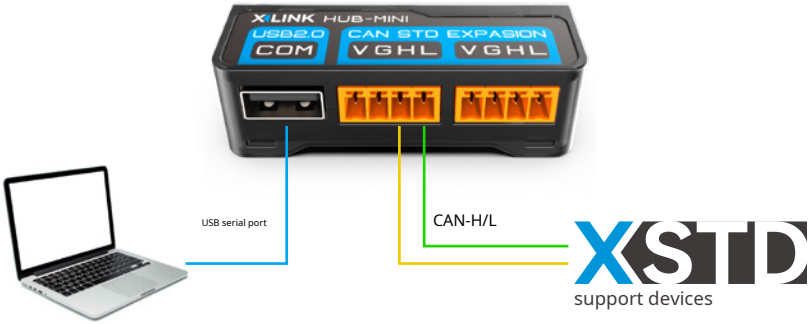
If you do not have a HUB, please contact customer serviceThe usage steps are as follows:

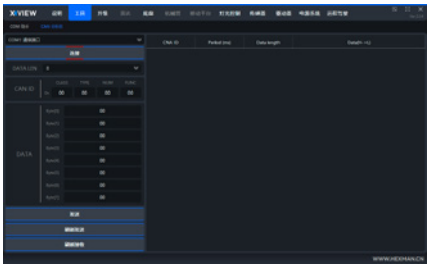
1. Download the driver package CP210x_Universal_Windows_Driver. https://www.silabs.com/documents/public/software/CP210x_Universal_Windows_Driver.zip

2: Connect the devices via the HUB module (see Figure 7.1 for details). The HUB module buzzer sounds three times. start up.

3. Clicking the Test Module button on the top edge of XVIEW will access the corresponding dedicated test function. If you need general test tools, please click the Tools button. XIVEW provides a serial port assistant and a CAN analyzer for use.(The CAN analyzer is only for use with devices that use extended frames.).

For detailed instructions on using XVIEW, please click the instructions button at the top of the software.





CAN analyzer

Serial Port Assistant

4.2 XROS Support

XROS is an open-source development kit based on ROS, helping users quickly use our various devices and products on the ROS platform, reducing the amount of large-scale development work. This code package supports the full set of XSTD license standards. Please contact customer service to obtain the development kit and usage instructions. Before use, you need to install the Ubuntu operating system and the ROS robot operating system yourself. This ROS package is compatible with... The following are the ROS and ROS2 versions. Please refer to the XROS User Manual for usage instructions.

Note: XROS requires a CAN-COM HUB as a conversion device. Please contact customer service for more technical support.

Ubuntu 18 Ubuntu 20	ROS Melodic Noetic	ROS2 Galactic Foxy

HEXMAN ROBOTICS

Qiyuan Robotics (Dongguan) Co., Ltd.

WWW.HEXMAN.CN

TEL: +86-0769-22678912 078112

MOBIL: +86-18652867127